The Mechanism Is Not the Eponym: Budget-Matched Ablation of Four Classical AI Method Families

Muhaiminul Islam Ninad
Department of Computer Science and Engineering
University of Dhaka
muhaiminulislam-2021111219@cs.du.ac.bd

July 2026

Abstract

Classical AI method families are named for a mechanism — the *heuristic* in heuristic search, the *constraints* in constraint propagation, the *swarm* in particle swarm optimisation, the *learning* in reinforcement learning — and both textbooks and practice credit that mechanism with the family's performance. We ask, for four families at once, whether the eponymous mechanism is the mechanism that does the work. We build four problems in a single application setting (urban mobility in Dhaka, Bangladesh), implement each family from scratch, and run ablations under strictly matched compute budgets, scoring each family on both its own customary metric and on a metric that charges every resource consumed.

In no family does the standard attribution survive the ablation intact, and in three of them the customary metric is what conceals the failure. **(i) Informed search:** the three "different" heuristics in a standard routing study are positive scalar multiples of one function, so greedy best-first search is provably invariant to the choice (confirmed: identical paths on 10/10 queries) and A* over them is a one-parameter weight sweep whose node counts and search times are perfectly rank-correlated with that scalar ($\rho = -1.0000$). Nodes-expanded ranks A* above uniform-cost search by 2.15×; wall clock ranks it *below* by 6.18×, because 91.6% of A*'s runtime is a one-off $O(|E|)$ heuristic calibration that the metric prices at zero. The fastest exact method in the study is uninformed. **(ii) Constraint satisfaction:** ordering and propagation reduce node counts exactly as folklore predicts (-3 to -5 nodes, $p \leq 10^{-4}$) while running 12–14× slower in wall clock and changing which instances are solvable in 1 case out of 80 — that one adversely; past the point where search stops completing, the node metric inverts and measures throughput rather than pruning. **(iii) Population-based search:** thirty non-communicating particles are statistically indistinguishable from random search ($p = 0.22$) and a single particle is *worse* than random search ($p < 10^{-4}$); switching on the social term alone, at the same budget and the same random stream, buys 7.6 minutes of every student's expected waiting time. The population is not the mechanism, the communication channel is. **(iv) Reinforcement learning:** on identical data — same seeds, same 400,000 transitions, same order — planning on an estimated model beats a *tuned* model-free learner (1.74% versus 2.72% regret; 89.9% versus 65.7% action-optimality), and planning with arrival rates wrong by a factor of ten still beats learning from scratch.

We give a common account of the four results: in each family the operative component is one whose cost is paid once and reused many times — an amortisable precomputation or a broadcast information channel — while the eponym's cost is paid per unit of work, and each

subfield's customary metric is precisely the one that declines to charge for reuse. We close with a metric-audit rule and release the testbed, code, and every number in this paper.

1 Introduction

A method family's name is a hypothesis about what makes it work. "Heuristic search" asserts that the heuristic is what turns an intractable search into a tractable one. "Constraint propagation" asserts that propagating constraints is what prunes the tree. "Particle *swarm* optimisation" asserts that the swarm — a population of interacting searchers — is the source of the optimisation power. "Reinforcement *learning*" asserts that learning from experience is the right response to not having a model.

These are good hypotheses. They are also rarely tested *as* hypotheses, because the natural experiment — remove the named mechanism, hold everything else fixed, and see what happens — requires a budget-matching discipline that the field's customary reporting metrics do not enforce and sometimes actively frustrate. Heuristic search reports nodes expanded, a metric that charges nothing for computing the heuristic. Metaheuristics report generations or iterations, metrics denominated in units whose cost varies with population size. Reinforcement learning reports sample complexity, a metric that charges nothing for what is done with a sample after it is collected. In each case the customary metric is the one under which the eponymous mechanism looks best, and this is not a coincidence: metrics are chosen by the communities that the mechanism defines.

This paper runs the natural experiment for four families at once. We construct four problems in a single application setting — urban mobility in Dhaka, a megacity for which almost no operational data is published — so that the four ablations share a domain, an implementation discipline, and an author. Each family is implemented from scratch, without solver libraries, so that the ablations can reach inside the method. Each comparison is budget-matched: identical objective-evaluation counts for the population methods, identical transition budgets for the learners, identical wall-clock budgets for the constraint solvers, and, for search, both the customary metric and a metric that charges every millisecond spent.

Findings. In each family the eponym is either not the operative mechanism, or is operative only under a metric that declines to charge what it costs. We state each result with its test in Sections 4–7. The headline in each case:

- **Search.** The heuristic "portfolio" in a conventional multi-heuristic routing study collapses to a single scalar, and the standard metric misprices the heuristic's cost by more than an order of magnitude. The fastest exact method we measure is an uninformed one.
- **Constraints.** Ordering and propagation prune as advertised and cost an order of magnitude more than they save, and once instances stop being solvable within budget the node metric measures the opposite of what it appears to. Repair-based search moves the cost of an attempt by three orders of magnitude while abandoning the mechanism the family is named for.
- **Population.** Neither the population nor the individual is doing the optimisation. A single shared best-so-far, broadcast once per iteration, is.
- **Learning.** The model is doing the work, not the learning; and the model can be grossly wrong and still win.

Contributions.

1. A budget-matched ablation protocol applied uniformly to four classical AI families (§3), with a from-scratch reference implementation of each.
2. Four ablation results (§4–§7), three of which disconfirm the standard attribution and one of which (§7) disconfirms it after we first *strengthen* the arm we expected to lose — a step that reduces our own headline effect sevenfold and which we report because omitting it would have been the more publishable choice.
3. A cross-family account of *why* the credit is misassigned (§8): in every case the operative component is one whose cost is amortisable across units of work, and every customary metric is one that does not charge for amortisation. We give a corresponding metric-audit rule.
4. A released, deterministic testbed: four problems, four reference implementations, and scripts that regenerate every table in this paper. Problems 3 and 4 carry 23 and 28 tests respectively, all passing with no skips; Problems 1 and 2 carry 7 and 18, and Problem 2's are not runnable from its shipped lockfile (§9).

What this paper is not. It is not a claim that heuristics, propagation, populations, or model-free learning are useless; all four are load-bearing under conditions we identify. It is not a claim about modern learned systems — every method here is tabular, exact, or small. And it is not a benchmark competition: no method here is proposed as state of the art. Section 9 is unusually long by design.

2 Related Work

Ablation as critique. A recurring genre establishes that a field's headline progress survives less well than reported once baselines are tuned and budgets matched: neural language models [Melis et al., 2018], deep reinforcement learning [Henderson et al., 2018], metric learning [Musgrave et al., 2020], and neural recommendation [Ferrari Dacrema et al., 2019]. Lipton and Steinhardt [2019] name a closely related failure — not identifying the true source of an empirical gain — among four troubling trends; the specific diagnoses of unmatched tuning budgets and weak baselines are made most directly by Sculley et al. [2018]. Our contribution differs in scope rather than in kind: instead of auditing one family's recent literature, we ablate the defining mechanism of four families simultaneously on one testbed, which lets us ask whether the misattributions share a structure. They do (§8).

Experimental algorithmics. That algorithm comparisons must charge all resources and report them honestly is old advice [Hooker, 1995, Johnson, 2002, McGeoch, 2012]. Hooker [1995] in particular argues for controlled experiments that isolate mechanisms rather than competitive testing of implementations, which is precisely the design here. Burns et al. [2012] document how far heuristic-search wall-clock results depend on implementation constants — a caution we take seriously in §9, since one of our results is a wall-clock reversal.

Search. A* [Hart et al., 1968] and its optimality theory [Dechter and Pearl, 1985, Pearl, 1984] are stated in terms of node expansions; weighted A* [Pohl, 1970] trades expansions for bounded suboptimality. Our §4 shows a standard multi-heuristic study reducing exactly to Pohl's

one-parameter family, and that the expansion-count metric that underwrites the theory misprices the heuristic in practice. Strong uninformed baselines — here bidirectional uniform-cost search, in the tradition of Holte et al. [2016] — are what make the reversal visible.

Constraint satisfaction. Forward checking [Haralick and Elliott, 1980] and arc consistency [Mackworth, 1977] define the propagation family; minimum-remaining-values and least-constraining-value orderings are its standard companions. Min-conflicts [Minton et al., 1992] is the repair-based alternative, historically presented as the surprising outsider. §5 finds the outsider dominating on every operational metric.

Population-based search. PSO [Kennedy and Eberhart, 1995] with inertia weighting [Shi and Eberhart, 1998], its topology literature [Kennedy and Mendes, 2002], and real-coded GAs with BLX-α [Eshelman and Schaffer, 1993] are all implemented here from their original update rules. Sörensen [2015] argues that metaphor-driven metaheuristics obscure their own mechanisms; §6 supplies a quantitative instance, isolating the social term as the entire source of PSO's advantage on our problem. Wolpert and Macready [1997] frames why such results must be stated relative to a problem class.

Decision making under uncertainty. Value iteration [Bellman, 1957, Puterman, 1994] and Q-learning [Watkins and Dayan, 1992, Sutton and Barto, 2018] are the model-based and model-free poles. The intermediate position — estimate a model from experience, then plan on the estimate — runs from Dyna [Sutton, 1990] through the PAC-MDP analyses [Kearns and Singh, 2002, Brafman and Tennenholtz, 2002] to certainty-equivalence results in control [Mania et al., 2019]. That model-based methods can be more sample-efficient is well established in theory; §7 contributes a controlled empirical instance in which the comparison is run on *byte-identical* data against a *tuned* model-free baseline, and in which the model is then degraded until it loses, locating the crossing point.

Domain. The road network is from OpenStreetMap via OSMnx [Boeing, 2017]. Our fixed-time signal baseline is Webster-*style* [Webster, 1958]: it splits a fixed green budget in proportion to mean arrival rates, but does not implement Webster's cycle-length optimisation, which is the substance of that paper. Delay accounting uses Little's law [Little, 1961].

3 Testbed and Protocol

3.1 Four problems, one setting

Table 1 summarises the testbed. The four problems were designed independently as instances of four method families, and share only their setting: mobility in Dhaka, and for the last two specifically the University of Dhaka campus. Sharing a setting is a convenience for exposition and a control on author effects; it is not a claim that the four problems are otherwise related. Problems 3 and 4 in particular are deliberately different in kind — an offline design problem over a continuous decision vector, and an online sequential control problem under uncertainty.

Problem 1 — multi-factor routing. Shortest-path queries on the Dhaka OpenStreetMap drive network (55,009 nodes, 137,529 directed edges) under a multi-factor edge cost that combines

Table 1: The four problems, the family each instantiates, the mechanism whose credit is under test, and the resource held fixed across arms.

Problem	Family	Eponym under test	Matched budget
Multi-factor routing	Informed search	the heuristic	per-query wall clock
Fuel-crisis allocation	Constraint satisfaction	ordering + propagation	5 s per instance
Shuttle timetabling	Population-based search	the population	3,030 objective evaluations
Signal control	Reinforcement learning	the learning	400,000 transitions

length, road class, congestion, surface quality, and a traveller-context term. Six search algorithms (uniform-cost, Dijkstra, bidirectional uniform-cost, A*, weighted A*, greedy best-first) run over three named heuristics, giving twelve configurations on each of ten randomly drawn landmark pairs.

Problem 2 — fuel-crisis allocation. Assigning n vehicles to (station, pump, time-slot) triples under fuel-type compatibility, vehicle range, pump exclusivity, station supply, and time windows, minimising a weighted sum of travel distance, waiting time, a quadratic lateness penalty for emergency vehicles, and unassigned-vehicle count. Five solvers: chronological backtracking; backtracking with minimum-remaining-values (MRV) and degree tie-break; with least-constraining-value (LCV); with forward checking plus MRV; and min-conflicts local search with a tabu list.

Problem 3 — shuttle timetabling. Choosing $K = 14$ continuous departure times in a 780-minute service window to minimise mean student waiting time, given bus capacity $C = 40$, fleet size $B = 3$, non-homogeneous Poisson arrivals with class-change bursts, congestion-dependent round-trip times, reneging after 60 minutes, and a quadratic fleet-overload penalty. The objective is non-differentiable, piecewise constant, and invariant under permutation of the decision vector (so every optimum has $14! \approx 8.7 \times 10^{10}$ symmetric copies). Solvers: PSO and a real-coded GA, both from scratch, plus four baselines.

Problem 4 — signal control. A finite discounted continuing MDP for a two-approach signalised intersection: state (q_A, q_B, ϕ, e, p) with queues truncated at 12, phase indicator, phase timer, and an exogenous traffic regime, giving $|\mathcal{S}| = 7,098$ and 11,154 legal state–action pairs under minimum- and maximum-green masks. Actions are hold and switch; switching burns a clearance tick; reward charges queue length, switching cost, and spillback. Solvers: value iteration, Q-learning, and certainty-equivalence planning, all from scratch, plus five baselines including Webster fixed-time.

3.2 Protocol

Budget matching is the load-bearing control. Within each family, every arm receives an identical allocation of the family's scarce resource (Table 1, last column). In Problem 3 this is enforced structurally: the objective is callable from exactly one function per solver file, that function increments a counter, and the counter is asserted equal to $30 \times (1 + 100) = 3030$ for every arm including every ablation. In Problem 4, the certainty-equivalence planner is handed the *literal transition list* that Q-learning generated — same seed, same transitions, same order —

so that any difference is attributable to what each method does with the data rather than to how much it received.

Determinism. Every experiment takes an explicit seed; there is no wall-clock or unseeded randomness in any solver. Re-running any experiment reproduces its table bit-for-bit — with the explicit exception of the wall-clock columns in §4 and §5, which are hardware- and load-dependent. Those we report as medians over repetitions, and the claims that rest on them are ratios, not absolute times. Where an arm is stochastic we report paired results over seeds and test them.

Statistics. Comparisons over seeds use the Wilcoxon signed-rank test [Wilcoxon, 1945] for location and Levene's test [Levene, 1960] for dispersion — specifically the Brown–Forsythe variant, which centres on the median rather than the mean — on 30 paired seeds for Problem 3 and 5 for Problem 4. We report p-values and make no claim of "better" without a test. We report disconfirmations of our own expectations in the same register as confirmations; three of the four sections below contain at least one.

Provenance. The four problems and their reference implementations originate as independent coursework assignments, each specified before its results were known and each carrying a written instruction to report disconfirming results rather than retune the instance. That origin is worth stating plainly: it explains the from-scratch implementations (no solver libraries were permitted), the per-problem test suites, and the fact that the four families were not selected after the fact to produce a common conclusion — the ablations in §6 and §7 were specified as experiments before either was run. It also bounds the scale: these are course-sized instances, not industrial benchmarks (§9). The cross-family reading in §8 is, by contrast, entirely post hoc.

Reporting only what runs. Every number in this paper was regenerated from the released code on the hardware in Appendix A: Tables 2 and 3 by the scripts in `repro/`, and the remainder by each problem's own experiment driver (`3/scripts/run.py`, `4/scripts/run.py`). Where a method is described in project documentation but is not present and executable in the released code, it is excluded from this paper entirely rather than reported from a stale artifact.

4 Informed Search: The Heuristic Is One Scalar, and Nodes Expanded Does Not Charge For It

4.1 The heuristic portfolio collapses to one parameter

The study defines three named heuristics — `admissible`, `fast`, and `realism` — and crosses them with three informed algorithms, in the standard pedagogical pattern of "one admissible heuristic and two inadmissible ones." All three have the form

$$h_i(n) \;=\; k_i \cdot d_{\text{geo}}(n, \text{goal}), \qquad k_i > 0, \tag{1}$$

where d_{geo} is great-circle distance and k_i is a constant depending on the graph, the traveller context, and the cost preset but *not* on n. Concretely, $k_{\text{adm}} = m^*$, the global minimum cost per metre; $k_{\text{fast}} = 1.25\, m^*$; and $k_{\text{realism}} = 1.08\, \bar{m}$ where $\bar{m}$ is the *mean* cost per metre. Measured on this graph, $m^* = 1.0113$ and $\bar{m} = 2.3213$, so $\bar{m}/m^* = 2.295$.

Two consequences follow immediately.

Proposition 1 (Greedy best-first is scale-invariant)**.** *Greedy best-first search orders its frontier by h alone. For $k > 0$, the orderings induced by h and $k\,h$ are identical. Hence greedy best-first with any two heuristics of the form* (1) *expands the same nodes in the same order and returns the same path.*

Proposition 2 (The informed configurations are one weight sweep)**.** A* *with $f = g + k_i\, d_{\text{geo}}$ is exactly weighted* A* *[Pohl, 1970] with $f = g + w_{\text{eff}} \cdot h_{adm}$ at weight $w_{\text{eff}} = k_i/m^*$; and weighted* A* *at explicit weight w over h_i is the same algorithm at $w_{\text{eff}} = w\,k_i/m^*$. The nine informed configurations therefore occupy only* seven *distinct points on a single scalar axis — the three greedy configurations coincide at $w_{\text{eff}} = \infty$, which is Proposition 1 — with uniform-cost search at $w_{\text{eff}} = 0$.*

Proposition 1 is confirmed exactly: across all ten queries, greedy best-first returns *byte-identical paths* under all three heuristics (10/10), expanding 120 nodes and incurring 30.754% suboptimality in every case. The three-heuristic portfolio is, for this algorithm, one heuristic.

Table 2 orders every configuration by w_{eff}. Both node count and search time are *perfectly* rank-correlated with the scalar (Spearman $\rho = -1.0000$ for each over the six distinct finite-weight configurations), and suboptimality is monotone non-decreasing ($\rho = +0.845$, with four exact ties at zero). What was presented as a 3×3 grid of algorithms and heuristics is a one-dimensional sweep of w_{eff}, and everything measured is a monotone function of it.

The algorithm axis has a duplicate in it too. Table 2 lists eleven rows, not twelve, because `dijkstra()` in this codebase is `return ucs(...)` with the result label overwritten (`algorithms.py:118--131`; its docstring says "same implementation as UCS here ... included as a separate entrypoint for coursework naming"). An earlier draft of this paper reported the two as separate configurations with different timings — 295.3 ms against 303.4 ms — which was pure timing noise on identical code presented as a comparison. We merge them. The study describes "six search algorithms"; it has five distinct ones behind six entry points. This is the same failure as the heuristic collapse, on the other axis of the grid, and our own audit rule (§8) did not initially catch it because we applied the rule only to the heuristics.

This is not a defect peculiar to this testbed; it is the default outcome of the standard instruction to supply "one admissible and two inadmissible heuristics" for a geometric domain, where the only readily available admissible quantity is a scaled straight-line distance and the natural way to make it inadmissible is to scale it further. The lesson is that a portfolio — of heuristics *or* of algorithms — must be checked for functional independence before it is crossed with anything, and that the check is cheap: compute the induced orderings, and diff the implementations.

4.2 Putting the arms on a common timing basis

Two measurement problems have to be fixed before any timing comparison here means anything, and one of them we got wrong in an earlier draft.

Setup is not search. Both heuristic constants m^* and $\bar{m}$ are global reductions over the edge set, computed by two full $O(|E|)$ passes that evaluate the multi-factor cost function on all 137,529 edges. The library's own clock excludes this (the internal timer starts after the aux build) while the batch driver's outer clock includes it. We report the two separately: setup

Table 2: All configurations ordered by effective weight w_{eff} (Prop. 2). Per (pair, configuration) we take the median of 5 repetitions, then the mean over ten landmark pairs. *Search* excludes heuristic setup and excludes the diagnostic post-processing discussed in §4.2; *setup* is the median of five isolated timings of `build_heuristic_aux`. *Spread* is the median run-to-run range as a fraction of the median. Gap is cost above optimal.

Configuration	w_{eff}	**Nodes**	**Search** (ms)	**Setup** (ms)	**Total** (ms)	**Gap** (%)	**Spread**
Uniform-cost search (= Dijkstra)	0	14,735	282.7	0.0	282.7	0.000	13%
Bidirectional UCS	0	7,316	155.4	0.0	**155.4**	0.000	15%
A* + admissible	1.000	6,845	146.1	1599.4	1745.6	0.000	11%
A* + fast	1.250	5,518	115.7	1599.4	1715.1	0.000	6%
Weighted A* + admissible	1.350	5,057	105.5	1599.4	1704.9	0.000	7%
Weighted A* + fast	1.688	3,540	72.4	1599.4	1671.8	0.000	7%
A* + realism	2.479	1,238	25.4	1599.4	1624.9	0.080	20%
Weighted A* + realism	3.347	492	10.1	1599.4	1609.5	3.222	19%
Greedy best-first + admissible	∞	120	2.5	1599.4	1601.9	30.754	21%
Greedy best-first + realism	∞	120	2.4	1599.4	1601.8	30.754	8%
Greedy best-first + fast	∞	120	2.5	1599.4	1601.9	30.754	7%

costs 1494–1819 ms across five isolated timings (median 1599 ms), against essentially zero for the uninformed methods.

The timed regions were not the same region. `astar()` computes `_suffix_costs()` — an $O(\text{path length}^2)$ re-evaluation of the multi-factor edge cost — plus a per-path-node heuristic evaluation *inside* its timed region, to produce the `heuristic_mean_abs_gap` diagnostic (`algorithms.py:288--303`). `ucs()`, `weighted_astar()` and `greedy_best_first()` do no post-processing at all. Comparing their reported times therefore compares different regions, and the asymmetry falls entirely on the arm we were measuring. It is worth a median of 26.0 ms on these paths (range 7.0–55.2 ms), which on the A* arm is not a rounding error. We time that post-processing on the identical path and subtract it, so Table 2 reports search-only time for every arm.

An earlier draft did not do this, and consequently reported A*'s per-node cost as 29.60 μs against uniform-cost search's 20.04 μs — a 1.48× per-node penalty attributed to heuristic evaluation. On the corrected basis the figures are 21.35 μs and 19.18 μs. The tell was visible in our own table and we missed it: weighted A*+admissible, which is the same algorithm at $w = 1.35$ and does no post-processing, came out at 23.2 μs per node, cheaper than plain A*. A heuristic cannot cost less to evaluate when you weight it more.

Repetitions. Search times on this machine vary run to run by 6–21% of the median (Table 2, last column) — enough that an earlier single-shot measurement of A*+admissible's search time (202.6 ms) and the median of five repetitions here (146.1 ms) differ by more than a third. All timings below are medians over five repetitions, and we quote ratios rather than absolute times wherever a claim rests on them.

4.3 The metric reversal

Node expansions are the customary metric of heuristic search, and by that metric A* with the admissible heuristic dominates uniform-cost search: 6,845 versus 14,735 expansions, a 2.15× reduction. By total wall clock the ranking reverses: 1745.6 ms versus 282.7 ms, a 6.18× *loss*. Setup is 91.6% of A*'s total runtime, and the node-count metric charges none of it.

Charging it correctly changes three conclusions — though, on the corrected basis, one of them by much less than we first claimed.

(i) Excluding setup, node counts overstate the heuristic's time benefit — but only by 1.11×. A* expands 2.15× fewer nodes and spends 1.93× less time searching (146.1 ms versus 282.7 ms), because an A* expansion costs 21.35 μs against uniform-cost's 19.18 μs. The principle stands — a node is a unit of cost only if the per-node cost is equal across arms, and introducing a heuristic violates that — but on this graph the violation is small: evaluating $k \cdot d_{\text{geo}}$ is a multiplication and a great-circle distance, which is cheap next to the multi-factor edge-cost evaluation that both arms pay on every generated edge. We previously reported this discrepancy as 1.48×; that figure was an artifact of our own instrumentation (§4.2), and the honest number is much less impressive. The interesting mispricing in this family is the setup, not the per-node cost.

(ii) The advantage is real and amortisable, and the break-even depends entirely on the baseline. The setup constant depends only on the graph, the traveller context, and the cost preset — not on the query — so hoisting it out of the query loop is legitimate for any workload that issues repeated queries in one context. Against uniform-cost search, A*+admissible saves 136.6 ms of search per query, so the 1599 ms setup pays for itself after 11.7 queries; weighted A*+fast breaks even after 7.6. Against bidirectional uniform-cost search the picture is very different: it searches in 155.4 ms, barely slower than A*+admissible's 146.1 ms, so that pairing needs 172 queries to amortise, while weighted A*+fast needs 19.3. A previous draft claimed A*+admissible *never* breaks even against bidirectional search; on the corrected timings that is wrong — it does, just two orders of magnitude later than against the weaker baseline.

(iii) The fastest exact method in the study is uninformed. Bidirectional uniform-cost search returns provably optimal paths on 10/10 queries in 155.4 ms total — 10.8× faster than the fastest exact informed configuration and 11.2× faster than A* with the admissible heuristic. It achieves a 2.01× node reduction over unidirectional uniform-cost search with no precomputation at all: a structural saving rather than an informational one, and the only one in this table that costs nothing to obtain.

4.4 Verdict

The heuristic in this study is one scalar. Its benefit is real, and on a per-node basis it is close to free (1.11×). What it is not is free *per context*: 91.6% of an informed query's runtime is an $O(|E|)$ calibration that the customary metric prices at zero, which reverses the ranking against uniform-cost search by 6.18× for a single query and takes 11.7 queries to repay. And for single-query exact routing the mechanism that actually wins is bidirectional expansion — structure, not information — which no amount of query volume is needed to justify.

Table 3: Left: outcomes over all 400 runs. Right: the 24 instances every solver solved, where node counts are uncensored and therefore meaningful; medians there, means on the left. Paired Wilcoxon against chronological backtracking, on node counts.

	all 400 runs			24 commonly solved instances		
Solver	**solved**	**nodes**	**time**	**nodes**	**time**	**vs. basic**
Chronological backtracking	27/80	444,111	3.15 s	13.5	0.09 ms	—
+ MRV / degree	27/80	322,997	3.02 s	13.5	0.14 ms	$+0.0$, $p = 0.27$
+ LCV	26/80	223,080	3.23 s	10.5	1.09 ms	-3.0, $p = 10^{-4}$
+ FC + MRV + deg.	27/80	126,481	3.17 s	9.0	1.27 ms	-5.0, $p < 10^{-4}$
Min-conflicts (repair)	25/80	**119**	**0.044 s**	**1.0**	**0.08 ms**	-13.5, $p < 10^{-4}$

5 Constraint Satisfaction: The Orderings Do Not Change What Gets Solved

Constraint satisfaction credits its performance to two mechanisms that share the family's name: *ordering* — choosing which variable to instantiate next (minimum remaining values, degree tie-break) and which value to try first (least constraining value) — and *propagation* — pruning domains after each assignment (forward checking). Both are taught as the difference between an exponential blow-up and a tractable search.

We ran five solvers — chronological backtracking, `bt_mrv`, `bt_lcv`, `bt_fc_mrv_deg` (forward checking + MRV + degree), and min-conflicts local search [Minton et al., 1992] — over 10 instance sizes × 8 seeds at a 5 s budget per instance: 400 runs. Every run records whether it exhausted the budget, which turns out to matter more than anything else in this section.

5.1 The orderings do not change what gets solved

Table 3 gives the outcome that the ordering literature would most expect to move, and it does not move. All four systematic solvers solve the same number of instances to within one, and pairwise they disagree on the outcome of **1 of 80 instances** — at $n = 25$, seed 303, where `bt_lcv` fails and the other three succeed. That single disagreement runs *against* the heuristic. Adding MRV, LCV, or forward checking to chronological backtracking changed which instances were solvable, at this budget, essentially not at all.

5.2 The orderings do reduce nodes — and make the solver slower

On the 24 instances every solver solved — the only regime where a node count means what it is supposed to mean — the ordering heuristics behave exactly as folklore predicts, and the improvement is statistically solid. Least-constraining-value removes a median of 3.0 nodes ($p = 10^{-4}$) and forward checking with MRV removes 5.0 ($p < 10^{-4}$), against a median of 13.5 for chronological backtracking. MRV on its own is the exception: a median difference of exactly zero ($p = 0.27$).

Every one of them is slower. LCV takes 12.1× and forward checking 14.1× the wall clock of chronological backtracking on the same instances (1.09 ms and 1.27 ms against 0.09 ms), and even MRV — which saves no nodes at all — costs 1.6×. The per-node price of computing the ordering and running the propagation exceeds, by an order of magnitude, the value of the nodes

it saves. This is the same structure as §4: the customary metric shows the eponymous mechanism working, and it *is* working, but the metric does not charge what it costs to run.

The one arm that improves both metrics simultaneously is the one that abandons systematic search. Min-conflicts expands a median of 1.0 node at 0.08 ms — as fast as chronological backtracking per attempt and 13.5 nodes cheaper — and over all 400 runs averages 119 nodes and 0.044 s against 444,111 nodes and 3.15 s: 3,700× fewer nodes and 72× less time, for 25 solved instances against 27. Both solve rates are low in absolute terms — 31% and 34% of 400 runs, because most instances in this sweep are out of reach for every solver at a 5 s budget — but min-conflicts reaches 93% of chronological backtracking's solve count for 0.03% of its search cost, by carrying one complete (initially infeasible) assignment forward and repairing it rather than rebuilding a partial assignment prefix on every backtrack.

5.3 Under censoring, the node metric inverts

At $n \geq 30$ every systematic run exhausts the 5 s budget and nothing is solved. Aggregate node counts over such runs are widely reported anyway — our own source artifact reports them — and they are worse than uninformative. When every solver stops at the same clock, its node count measures *how fast it grinds*, not how well it prunes:

Solver	$n = 30$	$n = 40$	$n = 50$
Chronological backtracking	163,065	179,949	168,470
+ MRV / degree	82,923	114,392	117,990
+ LCV	64,761	60,027	35,103
+ FC + MRV + degree	30,066	22,951	16,588

nodes per second on fully censored cells (all runs hit the budget; none solved)

Forward checking's apparent 3.5× node advantage in the all-runs column of Table 3 is therefore not a pruning result at all: it is lower throughput measured against a shared stopwatch, by 5.4× at $n = 30$, 7.8× at $n = 40$ and 10.2× at $n = 50$. Under censoring the sign of the metric's meaning flips — fewer nodes indicates a *slower* solver, not a better-pruned tree — and the two cases are indistinguishable in the reported number. Any scaling curve of nodes against n that crosses into a censored regime without saying so is reporting throughput and calling it efficiency.

5.4 Verdict

Ordering and propagation do what they claim to do, measured in the unit the field uses, on the instances where that unit is well defined. They also cost 12–14× more wall clock than they save at these sizes, change which instances are solvable in 1 case out of 80 (and that one adversely), and produce node counts at larger sizes that measure the opposite of what they appear to. The mechanism that actually changes the cost of solving these instances by orders of magnitude is the one the family is not named for.

Table 4: Communication ablation on the shuttle-timetabling objective J (minutes of mean student wait; lower is better). Identical 3,030-evaluation budget, 30 paired seeds. Wilcoxon signed-rank against random search.

Arm	**median J**	**mean J (sd)**	**vs. random search**
(a) 1 particle, whole budget alone	26.10	26.61 (2.92)	**worse**, $p = 7.1 \times 10^{-5}$
(b) 30 particles, $c_2 = 0$ (no sharing)	23.79	23.57 (1.76)	indistinguishable, $p = 0.22$
(c) 30 particles sharing one `gbest`	**15.91**	**16.23** (1.33)	better, $p < 10^{-8}$
(d) Random search (reference)	24.56	24.07 (1.61)	—

6 Population-Based Search: The Population Is Not the Mechanism

PSO's name attributes its power to a *swarm*: a population of searchers. The canonical velocity update

$$v \leftarrow \underbrace{w\,v}_{\text{momentum}} + \underbrace{c_1 r_1(\text{pbest} - x)}_{\text{cognitive}} + \underbrace{c_2 r_2(\text{social} - x)}_{\text{social}}, \qquad x \leftarrow x + v, \tag{2}$$

contains two candidate mechanisms — having many searchers, and letting them communicate — that the word "swarm" conflates. We separate them at a fixed budget of 3,030 objective evaluations, which every arm spends in exactly one instrumented function.

6.1 The ablation

Table 4 reports four arms over 30 paired seeds. Arm (b) sets $c_2 = 0$, severing communication while retaining thirty searchers; crucially r_2 is still drawn, so arms (b) and (c) consume the same random stream and differ only in the mechanism.

The answer is unusually clean. Thirty independent searchers are worth *nothing* over a single random sampler at the same budget: arm (b) versus arm (d) gives $p = 0.22$. The point estimates do nominally favour the non-communicating swarm (median 23.79 against 24.56; paired median difference -0.57 minutes), but the effect is not distinguishable from zero at 30 seeds, and it is an order of magnitude smaller than what communication buys. A single particle spending the entire budget alone is actively *worse* than random sampling ($p = 7.1 \times 10^{-5}$): with no social attractor, inertia walks it into a corner of the box and it stops sampling usefully. Only when the thirty searchers share a single best-so-far does the method work at all. Arms (b) and (c) differ in one coefficient and consume the identical random stream, so the paired median difference between them — 7.61 minutes, $p = 3.7 \times 10^{-9}$ — is attributable to the social term and to nothing else. In the units of the problem that is 7.61 minutes off every student's expected wait, bought by a single broadcast. Against random search the paired margin is 9.05 minutes ($p = 1.9 \times 10^{-9}$).

Neither "many searchers" nor "a searcher with memory" is the mechanism. The mechanism is a single shared scalar-plus-vector broadcast once per iteration. This is a quantitative instance of Sörensen's objection [Sörensen, 2015]: the biological metaphor names the population, and the population is the part that does not matter.

Table 5: Generalisation to 30 held-out arrival realisations (seeds 100–129). J in minutes. Service level is the percentage of students waiting $\leq 10\,min$.

Schedule	**train J**	**held-out J**	**optimism gap**	**service level**	**stranded**
PSO (best of 30 seeds)	14.23	**16.26**	2.03	47.1%	**8.8%**
GA (best of 30 seeds)	14.42	16.40	1.98	44.1%	7.9%
Demand-proportional	23.47	22.92	−0.54	46.3%	23.0%
Uniform headway	32.14	32.40	0.26	13.5%	9.6%

6.2 Three further disconfirmations

The GA beats the swarm on the swarm's own problem. At the identical budget, the real-coded GA reaches median $J = 15.15$ against PSO's 15.91 (Wilcoxon $W = 106$, $p = 0.008$, 30 paired seeds). We expected the reverse. The diagnosis is in the encoding and is specific: the objective is invariant under permutation of the departure-time vector, so gene *position* carries no meaning, while PSO's update (2) moves particle i's coordinate k toward the attractor's coordinate k — a correspondence that does not exist. BLX-α [Eshelman and Schaffer, 1993] has the same exposure but its per-gene interval sampling is less committed to the alignment, and Gaussian mutation supplies a local-search channel that PSO loses once inertia has decayed. Both methods nonetheless beat every baseline (Table 5).

The ring topology's celebrated steadiness is not present. Fully connected `gbest` beats the ring on location (mean J 16.23 vs 17.62; Wilcoxon $W = 59$, $p = 1.5 \times 10^{-4}$). The ring's variance is lower in the direction the literature predicts [Kennedy and Mendes, 2002] (1.10 vs 1.77), but Levene's test does not reject equality of variance ($p = 0.49$): on 30 seeds the difference is not established, and we decline to claim it. Both topologies were still improving when the budget expired (last improvement at iteration ≈ 99 of 100), so the binding constraint is the evaluation budget, not convergence — which also means this comparison says nothing about asymptotic behaviour.

The advantage survives out of sample, but not in the way the objective suggests. Schedules optimised on three arrival realisations were scored on 30 held-out realisations (Table 5). PSO beats the demand-proportional heuristic on *every one* of the 30 held-out days ($p < 10^{-8}$) with an optimism gap of only 2.03 minutes. But the metric a transport office would recognise — service level, the fraction of students waiting ≤ 10 minutes — is barely improved (47.1% vs 46.3%). What the optimiser actually buys is a collapse in *stranding*, from 23.0% to 8.8%: it carries the evening surge that the heuristic abandons. A study reporting only service level would have concluded the search was worthless.

The constraint that never costs anything. Across all 27 cells of a $K \in \{10, \ldots, 18\} \times B \in \{2, 3, 4\}$ sweep, the quadratic fleet-overload penalty at the found optimum is *exactly zero* and peak concurrency never exceeds B; at $B = 2$ it saturates at exactly 2. The optimiser routes around the constraint rather than paying for it, so the constraint is invisible in the objective and appears only as a shadow price. At the small end of the sweep the price is legible: cutting from three buses to two costs 1.61, 1.48 and 0.26 minutes of mean wait at $K = 10, 11, 12$, while a fourth bus is nominally *worse* there (−0.36 to −0.66 minutes). At $K \geq 13$ the pattern does

not hold — the fourth bus buys 1.37 minutes at $K = 13$ and 0.65 at $K = 14$ while the third buys nothing — and with seed standard deviations of 0.3 to 2.0 minutes, almost every one of these differences sits inside the noise. The defensible statement is the narrow one: below $K = 12$ a third bus is worth roughly a minute and a half of mean wait and a fourth is worth nothing, and above that our 5-seed cells cannot separate the fleet sizes at all. Reporting penalty terms alone would have reported a constraint that does not exist; reporting these differences without their standard deviations would have reported a procurement recommendation the data does not support.

7 Reinforcement Learning: The Model Is the Mechanism

7.1 The instance is sound, and every switch is anticipatory

Value iteration converges in 2,528 sweeps to $\epsilon = 10^{-8}$ with an empirical contraction rate of 0.98981 against the theoretical $\gamma = 0.99$ (absolute error 1.93×10^{-4}), and V^* agrees with the exact sparse linear solve $(I - \gamma P_\pi)V^\pi = R_\pi$ of its own greedy policy to 4.90×10^{-9}. Every policy in this section is scored the same way — by exact solution of that linear system under the *true* model — so the regret and action-optimality rankings carry no Monte-Carlo noise. The rollout statistics reported alongside them (switches per hour, mean delay) come from simulation and do. Regret is $100\,(V^* - V^\pi)/|V^*|$.

The instance has a property worth stating because it makes the lookahead claim provable rather than merely measured. Delay is charged on the queue *entering* the tick, so it is identical under both actions; holding discharges the green approach, weakly reducing expected spillback; and switching pays $c_{\text{switch}} \geq 0$ on top. Therefore $R(s, \mathsf{hold}) \geq R(s, \mathsf{switch})$ in every one of the 4,056 states with a genuine choice, with minimum immediate advantage exactly $+3.000 = c_{\text{switch}}$. Two consequences: the myopic ($\gamma = 0$) policy provably never switches unless maximum-green forces it, so it *coincides exactly* with always-hold (Table 6 confirms: identical to nine significant figures); and every voluntary switch in the optimal policy — 650 states — is bought *entirely* out of discounted future value, because the immediate term never once argues for it.

The mirror image locates where the reactive baseline loses: in 1,230 states the optimal policy holds the green while the *red* queue is longer, and longest-queue-first switches in every one of them. At $(q_A, q_B) = (11, 12)$ with green on A in peak, the reactive rule sees the side road one vehicle worse off and switches; V^* holds, because the main road is mid-discharge and the clearance tick would throw away five vehicles of capacity to save one vehicle of queue. The optimal switching curve is emphatically not the diagonal.

7.2 Planning on a learned model beats learning, on identical data

The decisive comparison holds the data fixed. Q-learning generates 400,000 transitions (4,000 episodes of 100 ticks, exploring starts, ϵ-greedy over legal actions, polynomial step size); the certainty-equivalence planner receives that *literal transition list* — same seed, same transitions, same order — estimates $\hat{P}$ and $\hat{R}$ from it, and plans to convergence on the estimate. Neither ever touches the true P or R; a test greps the learner's source for model access. State–action coverage is 68.1% of the 11,154 legal pairs, so this is a claim about a learner that saw two thirds of its problem.

Table 6: Regret and action-optimality are exact — solved from $(I - \gamma P_\pi)V^\pi = R_\pi$ under the true model, so they carry no Monte-Carlo noise. Switches/h and mean delay are rollout statistics from a single 50,000-tick simulation and do carry sampling noise; they are reported because they are the units a traffic engineer recognises, not because they rank policies. Myopic greedy and always-hold coincide by construction (see text).

Policy	**Regret**	**Action-opt.**	**Switches/h**	**Mean delay**
Value iteration (optimal)	0.0%	100.0%	84.2	26.9 s/veh
Longest-queue-first	9.9%	69.5%	97.0	29.0 s/veh
Fixed-time (Webster-style)	18.1%	33.3%	90.0	31.7 s/veh
Random (legal actions)	43.8%	50.1%	97.6	38.1 s/veh
Myopic greedy ($\gamma = 0$)	45.9%	84.0%	51.4	40.8 s/veh
Always hold	45.9%	84.0%	51.4	40.8 s/veh

Table 7: Same 400,000 transitions, same seeds, same order. 5 seeds, ± standard deviation.

Method	**Regret**	**Action-optimality**
Q-learning (default hyperparameters)	$12.49\% \pm 0.75$	65.7%
Q-learning (tuned)	$2.72\% \pm 0.23$	—
Certainty-equivalence VI (same data)	$\mathbf{1.74}\% \pm 0.41$	**89.9%**

The tuning step, and why we report it. The middle row of Table 7 is the most important row in this paper's methodology. A hyperparameter sweep (Appendix B) shows the *default* Q-learning configuration is far from the learner's best: initialising $Q_0 \approx \bar{V}^*$ instead of the nominally optimistic $Q_0 = 0$, together with a step exponent of 1.0 instead of 0.7, moves regret from 12.60% to 4.78% at the sweep's smaller budget — the initialisation being the larger share (12.60% → 5.51% on its own). $Q_0 = 0$ is optimistic here because every reward is negative, so $V^* < 0$ everywhere, and that optimism costs far more than it buys.

Running the comparison against the default learner would have let us report certainty equivalence beating Q-learning 12.49% to 1.74%: a sevenfold margin, a cleaner story, and an artifact of a handicapped baseline. Against the tuned learner the margin is 2.72% versus 1.74% — modest, real, and corroborated by the action-optimality column, where the gap is not modest at all (89.9% versus 65.7%). The honest claim is the smaller one, and it is the one we make: on identical data, planning on the estimated model converts the same transitions into substantially better decisions than learning the values directly.

The test, and the seed that goes the other way. Our protocol (§3) forbids a claim of "better" without a test, so: over the 5 paired seeds the difference is 0.98 percentage points with $t(4) = 3.87$, $p = 0.018$, $d_z = 1.73$. A two-sided Wilcoxon signed-rank test gives $p = 0.125$; at $n = 5$ that test cannot go below $p = 0.0625$ whatever the data, so it is not the informative choice here and we report it only for completeness. More important than either: **certainty equivalence wins on 4 of 5 seeds, not 5**. On seed 3 the tuned learner edges it, 2.3876% against 2.3893% — a margin of 0.0017 percentage points, which is a tie in everything but sign. We report it because a paper whose second contribution is the disconfirming step should not quietly round 4/5 up to a clean result. The action-optimality gap (89.9% vs 65.7%) is the more robust of the two comparisons and holds on every seed.

Why. The asymmetry is in what a sample buys. A transition spent on a Q-learning backup updates one state–action pair once, and value propagates backwards only along trajectories actually walked. The same transition spent on $\hat{P}$ and $\hat{R}$ becomes a permanent part of a model that unlimited subsequent planning sweeps read from for free — thousands of sweeps, every state, no further samples. Once the model is roughly right, extra planning is cheap and extra experience is not. This is the empirical shadow of the theory relating model estimation to near-optimal control [Kearns and Singh, 2002, Brafman and Tennenholtz, 2002, Mania et al., 2019].

7.3 How wrong may the model be?

If the model is what matters, the operational question is how wrong it may be before learning from scratch is preferable — a real question here, because Dhaka does not publish intersection arrival rates. We planned with assumed arrival rates scaled by $\lambda \in \{0, 0.1, \ldots, 2.0\}$ against the truth and compared to the tuned learner trained on 400,000 real transitions.

The crossing point is stark. Multipliers from 0.1 to 2.0 — rates wrong by a factor of ten too small or two too large — all yield regret $\leq 1.79\%$, beating the tuned learner's 2.72%. The *only* model that loses is $\lambda \times 0$: a planner that believes no vehicle ever arrives (3.58%). The crossing therefore sits between multipliers 0.0 and 0.1, which is to say:

> *The model has to be wrong about whether traffic exists at all.*

The reason is structural. The optimal switching curve is driven by the queue dynamics, the discharge rate, and the clearance cost — all of which the misspecified planner still has exactly right. The arrival rates only modulate how far ahead it is worth anticipating. Getting the *structure* right is what matters; getting the *rates* right barely does. That is a usable thing to be able to tell a transport office that has no arrival data and never will.

8 Cross-Family Synthesis

Four ablations, four misassignments. Taken singly each is a local curiosity. Taken together they share a structure.

8.1 The reuse asymmetry

In every case the component that turned out to be operative is one whose cost is *paid once and reused many times*, while the eponym's cost is paid per unit of work.

- **Search.** The heuristic's information is a global constant obtained by one $O(|E|)$ reduction. Its cost is per-context; its benefit is per-query. At one query it loses by 6.18×; past 12 queries against uniform-cost search, or 172 against bidirectional search, it wins. Node counting reports the $n \to \infty$ limit unconditionally, and never names either the assumption or the baseline it is relative to.
- **Constraints.** Systematic search reconstructs a partial assignment prefix on every backtrack; repair carries one complete assignment forward and reuses it across every step. The reused object is the assignment itself, which is why min-conflicts settles in a median of one node at 0.08 ms where forward checking spends nine at 1.27 ms.

Table 8: Each family's customary metric, the resource it does not charge, and the measured consequence in this paper.

Family	Customary metric	What it does not charge	Measured consequence
Informed search	nodes expanded	heuristic construction; per-node heuristic evaluation	ranking of A* vs. UCS reverses (2.15× win → 6.18× loss); setup is 91.6% of runtime
Constraint satisfaction	nodes, backtracks	per-node ordering and propagation cost; budget censoring	orderings save 3–5 nodes and cost 12–14× the wall clock; under censoring the metric measures throughput, inverting its sign
Population search	iterations, generations	evaluations per iteration; whether the population communicates	30 non-communicating particles ≡ random search ($p = 0.22$)
Reinforcement learning	samples, episodes	computation performed per sample	planning on $\hat{P}$ beats tuned Q-learning on identical data (1.74% vs 2.72%)

- **Population.** `gbest` is a broadcast channel: one evaluation, once found, informs all thirty particles for the remainder of the run. Sever it and thirty particles decay to thirty independent samplers, which is what the $p = 0.22$ against random search measures. The population supplies parallelism; only the channel supplies reuse.
- **Learning.** A transition spent on a Bellman backup is consumed once. The same transition spent on $\hat{P}$ is read by every subsequent planning sweep. Certainty equivalence wins because it converts a non-reusable resource into a reusable one.

Stated generally: *the mechanisms that survive budget-matched ablation are those that convert a per-unit cost into an amortised one.* This is not a law, and §9 lists cases where it would fail. But it predicts the four results here, and it explains the second pattern.

8.2 Each customary metric declines to charge for reuse

The metrics are not neutral. Each family's customary metric is denominated in exactly the unit that makes amortisation invisible (Table 8). Nodes expanded charges search work but not heuristic construction. Backtrack counts charge failure but not the cost of the machinery that avoids it. Iteration or generation counts are denominated per-population, so they hide how the population is used. Sample counts charge acquisition but not computation on what was acquired — which is the whole distinction between Q-learning and certainty equivalence.

8.3 A metric-audit rule

The four cases suggest a checklist that costs little and would have caught all of them:

1. **Name the unit.** State what one unit of the reported metric costs in wall clock, and verify that it is equal across arms. If a mechanism changes the per-unit cost — and heuristics,

propagators, and larger populations all do — the metric is not comparable across arms without reporting the per-unit cost too.

2. **Charge setup.** Report any per-context precomputation separately, and state the query volume at which it amortises. "A* is 2.15× better" and "A* pays off after 17 queries" are different claims; only the second is actionable.

3. **Ablate the eponym, not just the hyperparameters.** Sever the named mechanism at fixed budget while holding the random stream fixed. In §6 this cost one line ($c_2 = 0$, with r_2 still drawn) and produced the paper's cleanest result.

4. **Tune the arm you expect to lose.** §7 would have reported a sevenfold effect instead of a 1.6× one had we skipped this. Any ablation that only tunes the favoured arm measures tuning effort, not mechanism.

5. **Check *both* axes of the grid for independence.** Before crossing k heuristics with m algorithms, verify that the k heuristics induce distinct orderings and that the m algorithms are distinct code. Ours failed on both counts: the three heuristics were one heuristic under a scalar, and two of the six "algorithms" were the same function (§4). We applied this rule to the heuristic axis and not the algorithm axis, and it cost us a spurious row in our own table.

9 Threats to Validity

We take these seriously enough to enumerate them at length, because several of our results are reversals and reversals are fragile.

Implementation constants dominate one result. The wall-clock reversal in §4 is a statement about *this* implementation: pure Python, NetworkX adjacency, a cost function evaluated per edge without caching. Burns et al. [2012] show heuristic-search timings varying by an order of magnitude with implementation quality. A compiled implementation, or one that caches per-edge costs, would shrink the 1.57 s setup and move the break-even below 17 queries; a vectorised reduction could plausibly remove it almost entirely. What is *not* implementation-dependent is the structural point: the setup is $O(|E|)$ and the search is $O(\text{nodes expanded})$, these scale differently, and node counting prices the first at zero. The magnitude is ours; the asymmetry is not.

And one of those constants was ours to begin with. The more instructive failure is the one described in §4.2: `astar()` performed $O(\text{path length}^2)$ diagnostic post-processing inside its timed region and no other arm did, so an earlier draft of this paper compared different regions and inflated three of its own headline numbers. This is not covered by the implementation-constants caveat above, because it is not a constant factor — it is an instrumentation asymmetry that falls on exactly one arm. We caught it only when the per-node costs in our own table became internally inconsistent (weighted A* appearing cheaper per node than unweighted A* over the same heuristic). A paper arguing that fields mis-measure their own mechanisms should say plainly that it did so twice: here, and on the duplicate `dijkstra()` entry point.

Instances are synthetic. Every instance is generated, not measured. Parameters are motivated (class-change cadence, saturation flow of 1,800 veh/h, 60-minute reneging) but not calibrated

against field data, because the relevant field data for Dhaka is not published — which is itself part of the motivation for §7's misspecification study. Conclusions about *mechanisms* should transfer more readily than conclusions about *magnitudes*, but neither is established for real instances.

One setting, one author, one machine. All four problems share a domain and an author, which controls for some confounds and introduces others: a systematic blind spot in problem design would appear in all four. All timings are from a single laptop-class machine (Appendix A) with no isolation from OS scheduling; timing results should be read as ratios, not absolutes.

Statistical power. §4 uses 10 query pairs and reports means without tests. The headline effect is large (6.18×) and the invariance result is exact, but the timing means are medians over five repetitions with a run-to-run spread of 6–21%, and we report no confidence intervals on them. §6 uses 30 paired seeds, adequate for the reported effects but explicitly *inadequate* for the ring-variance question, which we therefore decline to claim. §7 uses 5 seeds. The 1.74% vs 2.72% gap is 2.96 pooled standard deviations, or $d_z = 1.73$ paired — the paired figure being the appropriate one, since the seeds are matched. An earlier draft reported "2.4 pooled standard deviations"; that number was the gap divided by the certainty-equivalence arm's standard deviation alone, and was wrong.

Scope of the collapse result. Proposition 2 applies to heuristics of the form $k \cdot d_{\text{geo}}$. It is not a claim that heuristic portfolios generally collapse; landmark-based, pattern-database, or learned heuristics are functionally independent by construction. The claim is that *this* portfolio collapsed, that the collapse was invisible in the study's own reporting, and that the check is cheap enough to be mandatory.

§5 measures a narrow band of instance sizes, and it is the easy band. This is the most serious limitation in the paper and we state it in the strongest form we can. The 24 instances every solver completed have a median search of 13.5 nodes and finish in under 1.3 ms. On searches that small, any per-node machinery loses on wall clock more or less by construction: forward checking's fixed cost per assignment cannot be repaid over nine nodes. Our finding that ordering and propagation cost 12–14× what they save is therefore a finding *about small instances*, and it is exactly the regime where the literature would not claim propagation helps. The honest scope is: propagation does not pay below ∼15 nodes of search, and our instances do not have a middle regime in which to test where it starts paying, because they jump from "solved in 13 nodes" to "not solved in five seconds" with almost nothing between. Establishing the crossover would need an instance generator with a tunable, gradual difficulty knob, which ours does not have. What survives regardless of size is the *censoring-inversion* result (§5.3), which is a statement about what a node count means when runs are truncated, not about which solver is better.

Censoring more generally. At $n \geq 30$ every systematic run exhausts its budget, so runtimes there are censored and carry no information about the solvers. We confine cross-solver runtime claims to the uncensored regime and report the large-n columns only to demonstrate the inversion; they are an anytime comparison, not a scaling law.

Certainty equivalence is favoured by the setting. §7's result holds in a tabular MDP with 7,098 states, where planning to convergence is free (3.8 s) and $\hat{P}$ can be stored exactly. It

also depends on a stated modelling choice — a pessimistic self-loop for unvisited state–action pairs — which is favourable to the planner at 68% coverage. Under function approximation, in large or continuous state spaces, or with adversarial coverage, the comparison could invert. We claim the result for this regime and note that the regime is the one most classical MDP pedagogy actually occupies.

Test coverage is uneven across the four problems. Problems 3 and 4 carry 23 and 28 tests and are the two whose algorithmic claims we lean on hardest; both suites pass with no skips, and both encode the load-bearing invariants directly (the budget-exactness assertion in §6, and a test that greps the Q-learner's own source to confirm it never reads the true model in §7). Problems 1 and 2 are weaker: 7 and 18 tests respectively, and Problem 2's suite is not runnable from its shipped lockfile at all, which does not list `pytest`. We verified Problem 2's solvers by re-running them and inspecting outputs rather than through its tests, and readers should weight §5 accordingly.

The synthesis is an interpretation. §8's reuse-asymmetry account is a post-hoc reading of four results, not a tested hypothesis. It generates predictions — e.g. that a PSO variant broadcasting more information per iteration should outperform `gbest` by more than the population size predicts — which we have not tested. Readers should weight it accordingly.

No claim of novelty for the individual mechanisms. That weighted A* trades nodes for suboptimality, that min-conflicts is fast, that social learning drives PSO, and that model-based methods can be sample-efficient are all known. The contribution is the budget-matched measurement of each on a common testbed, the metric-reversal magnitudes, and the cross-family structure — not the discovery of the underlying phenomena.

10 Conclusion

We asked whether the mechanism a classical AI method family is named for is the mechanism that produces its performance, and answered it four times under matched budgets. The heuristic in heuristic search turned out to be one scalar, priced at zero by the field's own metric, and beaten outright by an uninformed algorithm. The ordering heuristics in constraint satisfaction reordered search without changing what was solved. The swarm in particle swarm optimisation turned out to be a single broadcast channel, with the population itself statistically indistinguishable from random sampling once the channel was cut. And the learning in reinforcement learning lost, on its own data, to planning with a model estimated from that data — and kept losing until the model was wrong about whether traffic existed.

The common thread is that surviving mechanisms convert per-unit costs into amortised ones, and that each subfield's customary metric is the one that declines to charge for amortisation. That coincidence is worth more attention than any individual result here. The metric-audit rule in §8 is our attempt to make the check routine; the released testbed is our attempt to make it cheap.

Reproducibility. All code, data-generation scripts, and the scripts that regenerate every table in this paper are released at `https://github.com/ninadgns/eponym-ablation`. Every

experiment is seeded; every table reproduces exactly, except the wall-clock columns, whose ratios reproduce but whose absolute milliseconds are hardware-dependent.

References

Richard Bellman. *Dynamic Programming*. Princeton University Press, 1957.

Geoff Boeing. OSMnx: New methods for acquiring, constructing, analyzing, and visualizing complex street networks. *Computers, Environment and Urban Systems*, 65:126–139, 2017.

Ronen I. Brafman and Moshe Tennenholtz. R-MAX—a general polynomial time algorithm for near-optimal reinforcement learning. *Journal of Machine Learning Research*, 3:213–231, 2002.

Ethan Burns, Matthew Hatem, Michael J. Leighton, and Wheeler Ruml. Implementing fast heuristic search code. In *Proceedings of the Symposium on Combinatorial Search (SoCS)*, pages 25–32, 2012.

Rina Dechter and Judea Pearl. Generalized best-first search strategies and the optimality of A*. *Journal of the ACM*, 32(3):505–536, 1985.

Larry J. Eshelman and J. David Schaffer. Real-coded genetic algorithms and interval-schemata. In *Foundations of Genetic Algorithms 2*, pages 187–202. Morgan Kaufmann, 1993.

Maurizio Ferrari Dacrema, Paolo Cremonesi, and Dietmar Jannach. Are we really making much progress? a worrying analysis of recent neural recommendation approaches. In *Proceedings of the ACM Conference on Recommender Systems (RecSys)*, pages 101–109, 2019.

Robert M. Haralick and Gordon L. Elliott. Increasing tree search efficiency for constraint satisfaction problems. *Artificial Intelligence*, 14(3):263–313, 1980.

Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2):100–107, 1968.

Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. Deep reinforcement learning that matters. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2018.

Robert C. Holte, Ariel Felner, Guni Sharon, and Nathan R. Sturtevant. Bidirectional search that is guaranteed to meet in the middle. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2016.

John N. Hooker. Testing heuristics: We have it all wrong. *Journal of Heuristics*, 1(1):33–42, 1995.

David S. Johnson. A theoretician's guide to the experimental analysis of algorithms. In *Data Structures, Near Neighbor Searches, and Methodology*, pages 215–250. American Mathematical Society, 2002.

Michael Kearns and Satinder Singh. Near-optimal reinforcement learning in polynomial time. *Machine Learning*, 49(2–3):209–232, 2002.

James Kennedy and Russell Eberhart. Particle swarm optimization. In *Proceedings of the IEEE International Conference on Neural Networks*, pages 1942–1948, 1995.

James Kennedy and Rui Mendes. Population structure and particle swarm performance. In *Proceedings of the IEEE Congress on Evolutionary Computation*, pages 1671–1676, 2002.

Howard Levene. Robust tests for equality of variances. In *Contributions to Probability and Statistics: Essays in Honor of Harold Hotelling*, pages 278–292. Stanford University Press, 1960.

Zachary C. Lipton and Jacob Steinhardt. Research for practice: Troubling trends in machine-learning scholarship. *Communications of the ACM*, 62(6):45–53, 2019.

John D. C. Little. A proof for the queuing formula: $L = \lambda W$. *Operations Research*, 9(3):383–387, 1961.

Alan K. Mackworth. Consistency in networks of relations. *Artificial Intelligence*, 8(1):99–118, 1977.

Horia Mania, Stephen Tu, and Benjamin Recht. Certainty equivalence is efficient for linear quadratic control. In *Advances in Neural Information Processing Systems*, 2019.

Catherine C. McGeoch. *A Guide to Experimental Algorithmics*. Cambridge University Press, 2012.

Gábor Melis, Chris Dyer, and Phil Blunsom. On the state of the art of evaluation in neural language models. In *International Conference on Learning Representations*, 2018.

Steven Minton, Mark D. Johnston, Andrew B. Philips, and Philip Laird. Minimizing conflicts: A heuristic repair method for constraint satisfaction and scheduling problems. *Artificial Intelligence*, 58(1–3):161–205, 1992.

Kevin Musgrave, Serge Belongie, and Ser-Nam Lim. A metric learning reality check. In *Proceedings of the European Conference on Computer Vision (ECCV)*, 2020.

Judea Pearl. *Heuristics: Intelligent Search Strategies for Computer Problem Solving*. Addison-Wesley, 1984.

Ira Pohl. Heuristic search viewed as path finding in a graph. *Artificial Intelligence*, 1(3–4): 193–204, 1970.

Martin L. Puterman. *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. John Wiley & Sons, 1994.

D. Sculley, Jasper Snoek, Alex Wiltschko, and Ali Rahimi. Winner's curse? on pace, progress, and empirical rigor. In *International Conference on Learning Representations, Workshop Track*, 2018.

Yuhui Shi and Russell Eberhart. A modified particle swarm optimizer. In *Proceedings of the IEEE International Conference on Evolutionary Computation*, pages 69–73, 1998.

Kenneth Sörensen. Metaheuristics—the metaphor exposed. *International Transactions in Operational Research*, 22(1):3–18, 2015.

Richard S. Sutton. Integrated architectures for learning, planning, and reacting based on approximating dynamic programming. In *Proceedings of the International Conference on Machine Learning*, pages 216–224, 1990.

Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2nd edition, 2018.

Christopher J. C. H. Watkins and Peter Dayan. Technical note: Q-learning. *Machine Learning*, 8(3–4):279–292, 1992.

F. V. Webster. Traffic signal settings. Road Research Technical Paper 39, Road Research Laboratory, London, HMSO, 1958.

Frank Wilcoxon. Individual comparisons by ranking methods. *Biometrics Bulletin*, 1(6):80–83, 1945.

David H. Wolpert and William G. Macready. No free lunch theorems for optimization. *IEEE Transactions on Evolutionary Computation*, 1(1):67–82, 1997.

A Environment

All experiments: Intel Core i5-1135G7 @ 2.40 GHz (8 threads), 22 GB RAM, Linux 7.0.0, Python 3.12.6. Each problem is an independent `uv` project with a committed lockfile. Road network: OpenStreetMap extract of the Dhaka bounding box via OSMnx [Boeing, 2017], 55,009 nodes and 137,529 directed edges after simplification. Test suites: 23 tests (Problem 3) and 28 tests (Problem 4), no skips, both passing at the commit these results were generated from.

B Hyperparameter Sweep for §7

The sweep in Table 9 runs at a smaller budget than the headline comparison in Table 7 — 1,500 episodes (150,000 transitions) and 3 seeds rather than 4,000 episodes (400,000 transitions) and 5 seeds — so its absolute regrets are higher throughout and are not directly comparable to Table 7. It is used only to *select* the tuned configuration, which is then re-run at the full budget.

Three findings are worth stating.

Initialisation, not the step size, is the dominant lever. $Q_0 = 0$ looks neutral but is strongly *optimistic* in this MDP: every reward is negative, so $V^* < 0$ everywhere and a zero-initialised table over-values every unvisited action. Setting $Q_0 \approx \bar{V}^* = -560$ alone moves regret from 12.60% to 5.51% — more than any other single change, and more than the step-size exponent contributes.

The step-size schedule matters, in the direction theory predicts. At a fixed 150,000-transition budget the polynomial schedule $\alpha_n = (1+n)^{-p}$ substantially outperforms a constant step: constant $\alpha = 0.1$ and $\alpha = 0.5$ give 24.52% and 21.80% regret against 12.60% for the baseline schedule. Robbins–Monro conditions say a constant α will not converge asymptotically; that is *not* what this table measures, since every arm here stops at the same finite budget. What we observe is worse regret at a fixed budget, which is consistent with non-convergence but does not

demonstrate it. Within the polynomial family, larger p is better here ($p = 1.0$: 8.72%; $p = 0.7$: 12.60%; $p = 0.5$: 17.35%).

Exploring starts buy coverage, and coverage buys regret. Removing exploring starts collapses state–action coverage from 51.6% to 33.7% and degrades regret to 14.86%. Since coverage bounds what any tabular method can know, we report it beside every learning result rather than only reporting regret.

The γ rows are measured against each γ's own V^* and so are not comparable to the others; they are included to show that the learning problem gets harder as the effective horizon lengthens, which is the expected direction.

Table 9: Q-learning hyperparameter sweep. 13 configurations × 3 seeds at 150,000 transitions each. Baseline is $\alpha_n = (1+n)^{-0.7}$, ϵ floor 0.05, $Q_0 = 0$, exploring starts on.

Configuration	**Mean regret**	**sd**	**Coverage**
baseline	12.60%	1.46	51.6%
Step-size schedule			
α power 0.5	17.35%	0.93	51.6%
α power 1.0	8.72%	0.99	51.4%
constant $\alpha = 0.1$	24.52%	2.72	52.2%
constant $\alpha = 0.5$	21.80%	1.04	52.0%
Exploration			
ϵ floor 0.0	12.10%	0.86	51.5%
ϵ floor 0.2	11.65%	0.22	51.4%
ϵ floor 0.5	10.81%	1.00	50.6%
no exploring starts	14.86%	0.37	**33.7%**
Initialisation			
neutral $Q_0 = -560 \approx \bar{V}^*$	**5.51%**	0.36	49.8%
pessimistic $Q_0 = -2000$	9.96%	0.21	45.6%
tuned: $Q_0 = -560$, α power 1.0	**4.78%**	0.14	49.8%
Discount (each measured against its own V^*; not comparable above)			
$\gamma = 0.5$	3.41%	0.08	52.0%
$\gamma = 0.9$	8.80%	0.03	51.6%
$\gamma = 0.999$	13.92%	1.19	51.4%